```
ads: 8
      account: gitlab
      user: {username}
      role: {rolename}
      database: {databasename}
      warehouse: DEVXS
      schema: preparation
      authenticator: externalbrowser
    devl:
      type: snowflake
      threads: 16
      account: gitlab
      user: {username}
      role: {rolename}
      database: {databasename}
      warehouse: DEVL
      schema: preparation
      authenticator: externalbrowser
```

You open up your terminal and code editor, create a new branch, make an adjustment to a dbt model, and save your changes. You are ready to run dbt locally to test your changes, so you enter in the following command: dbt run --models @{modelname}. dbt starts building the models, and by default it builds them using the ANALYSTXS warehouse. After a while, the build fails due to a timeout error. Apparently the model tree you are building includes some large or complex models. In order for the queries to complete, you'll need to use a larger warehouse. You want to retry the build, but this time you want dbt to use the ANALYSTL warehouse instead of ANALYSTXS. So you enter in dbt run --models @{modelname} --target devl, which tells dbt to use the warehouse you specified in the devl target in your profiles.yml file. After a few minutes, the build completes and you start checking your work.

Venv Workflow {Venv-workflow}

Recommended workflow for anyone running a Mac system.

Using dbt

- Ensure you have the DBTPROFILEPATH environment variable set. This should be set if you've used the onboardingscript.zsh (recommened to use this as this latest and updated regularly), but if not, you can set it in your .bashrc or .zshrc by adding export DBTPROFILEPATH="/<yourrootdir/.dbt/" to the file or simply running the same command in your local terminal session
- Ensure that you have updated your .dbt/profiles.yml with your specific user configuration
- Ensure your SSH configuration is setup according to the GitLab directions. Your keys should be in ~/.ssh/ and the keys should have been generated with no password.
  - You will also need access to this project to run dbt deps for our main project.
- NB: Ensure your default browser is set to chrome. The built-in SSO login only works with chrome
- NB: Ensure you are in the folder where your /analytics repo is located. If you installed

everything properly jump analytics will land you where it is needed in order to run dbt commands successfully.
- NB: Before running dbt for the first time run make prepare-dbt. This will ensure you have venv installed.
  - This will run a series of commands including downloading and running a poetry install script.
  - If you get a certificate error like this urllib.error.URLError: <urlopen error SSL: CERTIFICATEVERIFYFAILED] certificate verify failed: unable to get local issuer certificate (ssl.c:1124)>, follow these [StackOverflow instructions.
- To start a dbt container and run commands from a shell inside it, use make run-dbt. This command will install or update the dependencies required for running dbt.
- To start a dbt container without the dependency update use make run-dbt-no-deps. This command assumes you already have the dbt dependencies installed. When using this command, if you make changes in any of the dependency packages (e.g. data-tests), you will need to run either dbt deps (from within the shell) or make run-dbt again for these changes to show up in your repository.
- This will automatically import everything dbt needs to run, including your local profiles.yml and repo files
- To see the docs for your current branch, run make run-dbt-docs and then visit localhost:8081 in a web-browser. Note that this requires the docs profile to be configured in your profiles.yml

Why do we use a virtual environment for local dbt development?

We use virtual environments for local dbt development because it ensures that each developer is running exactly the same dbt version with exactly the same dependencies. This minimizes the risk that developers will have different development experiences due to different software versions, and makes it easy to upgrade everyone's software simultaneously. Additionally, because our staging and production environments are containerized, this approach ensures that the same piece of code will execute as predictably as possible across all of our environments.

Build Changes Locally

To clone and build all of the changed models in the local development space the same buildchanges process can be used that is used in the CI Job. The primary difference is that instead of a WAREHOUSE variable the developer can pass a TARGET variable to use a target configured with a different warehouse size. To run the process, run the make build-changes command from within the virtual environment.

console
~/repos/analytics/transform/snowflake-dbt
··$ make build-changes DOWNSTREAM="+1" FULLREFRESH="True" TARGET="devxl" VARS="key":"valu
EXCLUDE="testmodel"

Video Introduction

SQLFluff linter

We use SQLFluff to enforce SQL style guide on our code. In addition to the methods for

executing the linter found in the documentation, when in the dbt virtual environment the make lint-models can be used. By default the lint-models process will lint all changed sql files, but the MODEL variable can be used to lint a specif sql file and the FIX variable can be used to run the linters fix command that will make changes to the sql file.

console
~/repos/analytics/transform/snowflake-dbt
··$ make lint-models
sqlfluff lint models/workspaces/workspacedata/mock/datatypemocktable.sql

~/repos/analytics/transform/snowflake-dbt
··$ make lint-models FIX="True" MODEL="dimdate"
sqlfluff fix ./models/common/dimensionsshared/dimdate.sql

Video Introduction

SAFE Check Locally

To test for SAFE coverage in model the same safemodelscript process can be used that is used in the CI Job. To run the process, run the make safe-check command from within the virtual environment.

console
~/repos/analytics/transform/snowflake-dbt
··$ make safe-check

Cloning models locally

This command enables zero copy cloning using dbt selections syntax to clone entire lineages. This is far faster and more cost-effective than running the models using dbt but do not run any dbt validations. As such, all dbt users are encouraged to use this command to set up your environment.

Prerequisites:

- Ensure you have dbt setup and are able to run models
- These local commands run using the SnowFlake user configured in /.dbt/profiles.yml, and will skip any table which your user does not have permissions to.
- These commands run using the same logic as the dbt CI pipelines, using the DBTMODELS as a variable to select a given lineage.
- You need to be in the /analytics directory to run these commands.

Usage:

To use the new clone-dbt-select-local-user-noscript command, you have to specify a DBTMODELS variable. For example, to clone only the dimsubscription model, you would execute the following command:

```
console
make DBTMODELS="dimsubscription" clone-dbt-select-local-user-noscript
```

This will clone the dbt model from the prod branch into your local user database (i.e., {username}PROD). You can use dbt selectors: @, +, etc to select the entire lineage that you want to copy over your local database.

Tips:

- If you encounter an error as below:

```
console
Compilation Error
  dbt found 7 package(s) specified in packages.yml, but only 0 package(s)
  installed in dbtpackages. Run "dbt deps" to install package dependencies.
```

- Run make dbt-deps from the root of the analytics folder and retry the command.

Transition Note:

We are actively transitioning to the new clone-dbt-select-local-user-noscript command. The old clone-dbt-select-local-user command will still be available for a limited time, but we encourage you to start using the new command as soon as possible.

Cloning into local user DB (python scripts - pre-dbt clone)

- This clones the given dbt model lineage into the active branch DB (ie. {username}PROD)
  - make DBTMODELS="<dbtselector>" clone-dbt-select-local-user
  - eg. make DBTMODELS="+dimsubscription" clone-dbt-select-local-user

Cloning into branch DB

- This clones the given dbt model lineage into the active branch DB (ie. {branchname}PROD), this is equivalent to running the CI pipelines clone step.
- It does not work on Master.
  - make DBTMODELS="<dbtselector>" clone-dbt-select-local-branch
  - eg. make DBTMODELS="+dimsubscription" clone-dbt-select-local-branch

Docker Workflow {docker-workflow}

The below is the recommended workflow primarily for users running Linux as the venv workflow has fewer prerequisites and is considerably faster.

To abstract away some of the complexity around handling dbt and its dependencies locally, the main analytics project supports using dbt from within a Docker container.
We build the container from the data-image project.
There are commands within the Makefile to facilitate this, and if at any time you have questions about the various make commands and what they do, use make help to get a list of the

commands and what each of them does.

Before your initial run (and whenever the containers get updated) make sure to run the following commands:

1. make update-containers
1. make cleanup

These commands will ensure you get the newest versions of the containers and generally clean up your local Docker environment.

Using dbt

- Ensure you have the DBTPROFILEPATH environment variable set. This should be set if you've used the onboardingscript.zsh (recommened to use this as this latest and updated regularly) or onboardingscript.sh, but if not, you can set it in your .bashrc or .zshrc by adding export DBTPROFILEPATH="/<yourrootdir/.dbt/" to the file or simply running the same command in your local terminal session
- Ensure that you have updated your .dbt/profiles.yml with your specific user configuration
- Ensure your SSH configuration is setup according to the GitLab directions. Your keys should be in ~/.ssh/ and the keys should have been generated with no password.
  - You will also need access to this project to run dbt deps for our main project.
- To start a dbt container and run commands from a shell inside of it, use make dbt-image
- This will automatically import everything dbt needs to run, including your local profiles.yml and repo files
  - You may see some WARNINGS about missing variables (GITBRANCH, KUBECONFIG, GOOGLEAPPLICATIONCREDENTIALS, etc.). Unless you are developing on Airflow this is ok and expected.
- To see the docs for your current branch, run make dbt-docs and then visit localhost:8081 in a web-browser. Note that this requires the docs profile to be configured in your profiles.yml
- Once inside of the dbt container, run any dbt commands as you normally would
- Changes that are made to any files in the repo will automatically be updated within the container. There is no need to restart the container when you change a file through your editor!

Command line cheat sheet

This is a simplified version of the primary command reference.

dbt specific:

- dbt clean - this will remove the /dbtmodules (populated when you run deps) and /target folder (populated when models are run)
- dbt run - regular run
- Model selection syntax (source). Specifying models can save you a lot of time by only running/testing the models that you think are relevant. However, there is a risk that you'll forget to specify an important upstream dependency so it's a good idea to understand the syntax thoroughly:
  - dbt run --models modelname - will only run modelname
  - dbt run --models +modelname - will run modelname and all parents

  - dbt run --models modelname+ - will run modelname and all children
  - dbt run --models +modelname+ - will run modelname, and all parents and children
  - dbt run --models @modelname - will run modelname, all parents, all children, AND all parents of all children
  - dbt run --exclude modelname - will run all models except modelname
  - Note that all of these work with folder selection syntax too:
    - dbt run --models folder - will run all models in a folder
    - dbt run --models folder.subfolder - will run all models in the subfolder
    - dbt run --models +folder.subfolder - will run all models in the subfolder and all parents
- dbt run --full-refresh - will refresh incremental models
- dbt test - will run custom data tests and schema tests; TIP: dbt test takes the same --model and --exclude syntax referenced for dbt run
- dbt seed - will load csv files specified in the data-paths directory into the data warehouse. Also see the seeds section of this guide
- dbt compile - compiles the templated code within the model(s) and outputs the result in the target/ folder.
    This isn't a command you will need to run regularly as dbt will compile the models automatically when you to 'dbt run'.
    One common use-case is the compiled code can be run in Snowflake directly for debugging a model.

    Works only if you've run the onboarding script:
- dbtrunchanged - a function we've added to your computer that only runs models that have changed (this is accessible from within the docker container)
- cyclelogs - a function we've added to your computer to clear out the dbt logs (not accessible from within the docker container)
- make dbt-docs - a command that will spin up a local container to serve you the dbt docs in a web-browser, found at localhost:8081

VSCode extension: dbt Power User

dbt Power User makes VScode seamlessly work with dbt. The guide below will allow you to install dbt Power User if you followed the Venv workflow.

Before we start, there are some settings to adjust in your VScode:

- Go in Code > Settings > Settings·
  - Search for 'Python info visibility' > Set this setting as 'Always'
  - In a terminal, run make run-dbt as described in the Using dbt section. Once it ran and the new shell spawned, run echo $VIRTUALENV. Copy that value.
    - Search for 'venv path' in VScode settings.
    - Set this setting to the path that you copied last step, which should look like /Users/<username>/Library/Caches/pypoetry/virtualenvs/ if you followed a standard installation. Remove the last part of the path analytics--py3.10 at the time of writing.
- Open VScode in /analytics (File > Open Folder... or Workspace...)
- You now see a python interpreter selector at the bottom right of VScode, click on it
  - In the popup field, you should now see the analytics venv(s) shown as type poetry, and the one used for dbt. Select it.
- Install extension dbt-power-user
- Follow only step How to setup the extension > Associate your .sql files the jinja-sql

language here

- In VScode, go back to the File view and find the analytics/.vscode/settings.json file that was created when you opened /analytics with vscode (create .vscode/settings.json if you cannot find it). This file will define the settings of VScode when opened in /analytics

Add this in your settings.json file:

```
console
{
"terminal.integrated.env.osx": {
"SHELL":"/bin/zsh",
"DBTPROFILESDIR": "../../../.dbt/",
"DATATESTBRANCH":"main",
"SNOWFLAKEPRODDATABASE":"PROD",
"SNOWFLAKEPREPDATABASE":"PREP",
"SNOWFLAKESNAPSHOTDATABASE":"SNOWFLAKE",
"SNOWFLAKELOADDATABASE":"RAW",
"SNOWFLAKESTATICDATABASE":"STATIC",
"SNOWFLAKEPREPSCHEMA":"preparation",
"SNOWFLAKETRANSFORMWAREHOUSE":"ANALYSTXS",
"SALT":"pizza",
"SALTIP":"pie",
"SALTNAME":"pepperoni",
"SALTEMAIL":"cheese",
"SALTPASSWORD":"416C736F4E6F745365637265FFFFFFAB"
},
"dbt.queryLimit": 500
}
```

Note: If the code base is updated with new values for these environment variables, you will have to update them in your settings.json according to the values of variables located in the Makefile at the root of the analytics repository.

- Edit DBTPROFILESDIR so that it points to your ~/.dbt/ folder (it seems that path must be relative and pointing to your ~/.dbt folder, from the /analytics folder)
- Restart VScode and re-open the analytics workspace
- Check that dbt-power-user is running by navigating through models with Command+click (dbt needs some time to init).
Ignore any warnings about dbt not up to date.
- Open a random model, right click in the sql code, Click run dbt model and check for output.

If you are getting an error of the type:

```
console
dbt.exceptions.RuntimeException: Runtime Error
  Database error while listing schemas in database ""PRODPROD""
  Database Error
    002043 (02000): SQL compilation error:
```

Object does not exist, or operation cannot be performed.

- then change the target profile used by dbt: navigate to dbt-power-user extension settings (Extensions > dbt-power-user > cog > Extension settings) and edit the setting called Dbt: Run Model Command Additional Params (same for build)

{{% panel header="Note" header-bg="warning" %}}
When running/building/testing a model from VS code UI, the terminal window popping is only a log output. Cmd+C does not stop the job(s), nor clicking the Trash icon in VS code. If you want to stop a job started via VScode, go through the Snowflake UI and your job list and kill the job(s) from there.
{{% /panel %}}

Configuration for contributing to dbt project

If you're interested in contributing to dbt, here's our recommended way of setting up your local environment to make it easy.

- Fork the dbt project via the GitHub UI to your personal namespace
- Clone the project locally
- Create a virtual environment (venv) for dbt following these commands

```
bash
cd ~
mkdir .venv  This should be in your root "~" directory
python -m venv .venv/dbt
source ~/.venv/dbt/bin/activate
pip install dbt
```

- Consider adding alias dbt!="source ~/.venv/dbt/bin/activate" to your .bashrc or .zshrc to make it easy to start the virtual environment
- Navigate to the dbt project in the same terminal window - you should see (dbt) at the start of the command prompt
- Run pip install -r editablerequirements.txt. This will ensure when you run dbt locally in your venv you're using the code on your machine.
- Run which dbt to ensure it's pointing to the venv
- Develop code locally, commit your changes as you would, and push up to your namespace on GitHub

When you're ready to submit your code for an MR, ensure you've signed their CLA.

Style and Usage Guide

Model Structure

As we transition to a more Kimball-style warehouse, we are improving how we organize models in the warehouse and in our project structure.
The following sections will all be top-level directories under the models directory, which is a

dbt default.
This structure is inspired by how dbt Labs structures their projects.

{{% panel header="Legacy Structure" header-bg="warning" %}}
Prior to our focus on Kimball dimensional modeling, we took inspiration from the BEAM\ approach to modeling introduced in "Agile Data Warehouse Design" by Corr and Stagnitto.
Many of the existing models still follow that pattern.
The information in this section is from previous iterations of the handbook.

- The goal of a (final) xf dbt model should be a BEAM table, which means it follows the business event analysis & model structure and answers the who, what, where, when, how many, why, and how question combinations that measure the business.
- base models- the only dbt models that reference the source table; base models have minimal transformational logic (usually limited to filtering out rows with data integrity issues or actively flagged not for analysis and renaming columns for easier analysis); can be found in the legacy schema; is used in ref statements by end-user models
- end-user models - dbt models used for analysis. The final version of a model will likely be indicated with an xf suffix when it's goal is to be a BEAM table. It should follow the business event analysis & model structure and answer the who, what, where, when, how many, why, and how question combinations that measure the business. End user models are found in the legacy schema.

Look at the Use This Not That mapping to determine which new Kimball model replaces the legacy model.
{{% /panel %}}

{{% panel header="FY21-Q4 Model Migration" header-bg="success" %}}
In FY21-Q4 the prod and prep databases were introduced to replace the analytics database. These 2 new databases will fully replace the analytics database.

Local development was also switched from custom schemas to custom databases.
{{% /panel %}}

Sources

All raw data will still be in the RAW database in Snowflake.
These raw tables are referred to as source tables or raw tables.
They are typically stored in a schema that indicates its original data source, e.g. netsuite

Sources are defined in dbt using a sources.yml file.

- We use a variable to reference the database in dbt sources, so that if we're testing changes in a Snowflake clone, the reference can be programmatically set
- When working with source tables with names that don't meet our usual convention or have unclear meanings, use identifiers to override source table names when the original is messy or confusing. (Docs on using identifiers)

yaml
Good
tables:

```
  - name: bizibleattributiontouchpoint
    identifier: bizible2bizibleattributiontouchpointc

 Bad
tables:
  - name: bizible2bizibleattributiontouchpointc
```

Source models

We are enforcing a very thin source layer on top of all raw data.
This directory is where the majority of source-specific transformations will be stored.
These are "base" models that pull directly from the raw data and do the prep work required to make facts and dimensions and should do only the following:

- Rename fields to user-friendly names
- Cast columns to appropriate types
- Minimal transformations that are 100% guaranteed to be useful for the foreseeable future. An example of this is parsing out the Salesforce ID from a field known to have messy data.
- Placement in a logically named schema

Even in cases where the underlying raw data is perfectly cast and named, there should still exist a source model which enforces the formatting.
This is for the convenience of end users so they only have one place to look and it makes permissioning cleaner in situations where this perfect data is sensitive.

The following should not be done in a source model:

- Removing data
- Joining to other tables
- Transformations that fundamentally alter the meaning of a column

For all intents and purposes, the source models should be considered the "raw" data for the vast majority of users.

Key points to remember:

- These models will be written to a logically named schema in the prep database based on the data source type. For example:
  - Zuora data stored in raw.zuora would have source models in prep.zuora
  - GitLab.com data with tables stored in raw.tappostgres.gitlabdb would have source models in prep.gitlabdotcom
  - Customers.gitlab.com data with tables stored in raw.tappostgres.customersdb would have source models in prep.customersdb
- These models should be organized by source - this will usually map to a schema in the raw database
- The name of source models should end with source
- Only source models should select from source/raw tables
- Source models should not select from the raw database directly. Instead, they should reference sources with jinja, e.g. FROM {{ source('workday', 'jobinfo') }}

- Only a single source model should be able to select from a given source table
- Source models should be placed in the /models/sources/<datasource/ directory
- Source models should perform all necessary data type casting, using the :: syntax when casting (You accomplish the same thing with fewer characters, and it presents as cleaner).
  - Ideally, source models should cast every column. Explicit is better than implicit. Test your assumptions
- Source models should perform all field naming to force field names to conform to standard field naming conventions
- Source fields that use reserved words must be renamed in source models
- Source models for particularly large data should always end with an ORDER BY statement on a logical field (usually a relevant timestamp). This essentially defines the cluster key for the warehouse and will help to take advantage of Snowflake's micro-partitioning.
- Exception: occasionally a data source is only useful when two sources are combined. If this is the case then we can join them in the source model; this is done for several Clari source models, i.e clarifieldssource

For a visual of how the source models relate to the raw tables and how they can act as a clean layer for all downstream modeling, see the following chart:

```
mermaid
graph LR

  subgraph "RAW Database"

    subgraph "tappostgres Schema"
      GitLabRaw[GitLab Users Raw Table ]
      VersionRaw[Version DB Pings Raw Table ]
    end

    subgraph "zuora Schema"
      ZuoraRaw[Zuora Account Raw Table ]
    end

  end

  subgraph "PREP Database"

    subgraph "gitlabdotcom Schema"
      GitLabRaw --> gitlabusers[GitLab Users Source Model ]
    end

    subgraph "versiondb Schema"
      VersionRaw --> versionpings[Version DB Pings Source Model ]
    end

    subgraph "zuora Schema"
      ZuoraRaw --> ZuoraAccount[Zuora Account Source Model ]
    end

    subgraph "preparation Schema"
```

```
    dbtModels[More dbt models ]
    gitlabusers --> dbtModels
    versionpings --> dbtModels
    ZuoraAccount --> dbtModels
  end

end

subgraph "PROD Database"

  subgraph "common schema"
    dbtModels --> dbtModels2[More dbt models ]
    gitlabusers --> dbtModels2
    versionpings --> dbtModels2
    ZuoraAccount --> dbtModels2
  end

  subgraph "commonmapping schema"
    dbtModels --> dbtModels3[More dbt models ]
    gitlabusers --> dbtModels3
    versionpings --> dbtModels3
    ZuoraAccount --> dbtModels3
  end

end
```

Sensitive Data

In some cases, there are columns whose raw values should not be exposed. This includes things like customer emails and names. There are legitimate reasons to need this data however, and the following is how we secure this data while still making it available to those with a legitimate need to know.

Static Masking

For a given model using static masking, the source format is followed as above. There is no hashing of columns in the source model. This should be treated the same as the raw data in terms of security and access.

Sensitive columns are documented in the schema.yml file using the meta key and setting sensitive equal to true. An example is as follows.

yaml

```
  - name: sfdccontactsource
    description: Source model for SFDC Contacts
    columns:
        - name: contactid
          tests:
            - notnull
```

```
      - unique
    - name: contactemail
      meta:
        sensitive: true
    - name: contactname
      meta:
        sensitive: true
```

Two separate models are then created from the source model: a sensitive and non-sensitive model.

The non-sensitive model uses a dbt macro called hashsensitivecolumns which takes the source table and hashes all of the columns with sensitive: true in the meta field. There is no specific join key specified since all columns are hashed in the same way. Another column can be added in this model as a join key outside of the macro, if needed. The sfdccontact model is a good example of this. 2 columns are hashed but an additional primary key of contactid is specified.

In the sensitive model, the dbt macro nohashsensitivecolumns is used to create a join key. The macro takes the source table and a column as the join key and it returns the hashed column as the join key and the remaining columns unhashed. The sfdccontactpii model is a good example of the macro in use.

All hashing includes a salt) as well. These are specified via environment variables. There are different salts depending on the type of data. These are defined in the getsalt macro and are also set when using the dbt container for local development.

In general, team members should not be permitted to see the salt used in the query string within the Snowflake UI. In table models this goal is met by using the Snowflake built-in ENCRYPT function. For models materialized into views, the ENCRYPT function seems to not work. Instead, a workaround using secure views is utilized. A secure view limits DDL viewing to the owner only, thus limiting visibility of the hash. To create a secure view, set secure equal to true in the model configuration. A view that utilizes the hashing functionality as described, but is not configured as a secure view, will likely not be queryable.

Dynamic Masking

When the sensitive data needs to be known by some users but not all then dynamic masking can be applied.

Sensitive columns to be masked dynamically are documented in the schema.yml file using the meta key and setting maskingpolicy equal to one of the Data Masking Roles found in the roles.yml file. An example is as follows.

yaml

```
  - name: sfdccontactsource
    description: Source model for SFDC Contacts
    columns:
        - name: contactid
```

```
      tests:
        - notnull
        - unique
    - name: contactemail
      meta:
        maskingpolicy: generaldatamasking
    - name: contactname
      meta:
        maskingpolicy: generaldatamasking
```

A post-hook running the macro maskmodel will need to be configured for any model that will need dynamic masking applied.

The maskmodel macro will first retrieve all of the columns of the given model that have a maskingpolicy identified. That information is passed to an other macro, applymaskingpolicy, witch orchestrates the creation and application of Snowflake masking policies for the given columns.

The first step of the applymaskingpolicy is to get the data type of the columns to be masked as the polices are data type dependant. This is done with a query to the data base informationschema table with the following query:

```
sql
SELECT
  t.tablecatalog,
  t.tableschema,
  t.tablename,
  t.tabletype,
  c.columnname,
  c.datatype
FROM "{{ database }}".informationschema.tables t
INNER JOIN "{{ database }}".informationschema.columns c
  ON c.tableschema = t.tableschema
  AND c.tablename = t.tablename
WHERE t.tablecatalog =  '{{ database.upper() }}'
  AND t.tabletype IN ('BASE TABLE', 'VIEW')
  AND t.tableschema = '{{ schema.upper() }}'
  AND t.tablename = '{{ alias.upper() }}'
ORDER BY t.tableschema,
  t.tablename;
```

Where the database, schema, and alias are passed to the macro at the time it is called.

With the qualified column name and the data type, masking policies are created for a given database, schema, and data type for the specified masking policy. And once the policies have been created they are applied to the identified columns.

It should be noted that permissions are based on an allow list of roles. Meaning permission has

to be granted to see the unmasked data at query time:

```
sql
CREATE OR REPLACE MASKING POLICY "{{ database }}".{{ schema }}.{{ policy }}{{ datatype }} AS
(val {{ datatype }})
  RETURNS {{ datatype }} ->
     CASE
       WHEN CURRENTROLE() IN ('transformer','loader') THEN val  -- Set for specific roles that
should always have access
       WHEN ISROLEINSESSION('{{ policy }}') THEN val -- Set for the user to inherit access
bases on there roles
       ELSE {{ mask }}
     END;
```

Only one policy can be applied to a column so users that need access will have to have the permissions granted using the applied masking role.

Staging

Prior to our implementation of Kimball modeling, most all of our models would have fallen into Staging category.

This directory is where the majority of business-specific transformations will be stored. This layer of modeling is considerably more complex than creating source models, and the models are highly tailored to the analytical needs of business.
This includes:

- Filtering irrelevant records
- Choosing columns required for analytics
- Renaming columns to represent abstract business concepts
- Joining to other tables
- Executing business logic
- Modelling of fctand dim tables following Kimball methodology

Workspaces

We provide a space in the dbt project for code that is team specific and not meant to adhere to all of the coding and style guides. This is in an effort to help teams iterate faster using solutions that don't need to be more robust.

Within the project there is a /models/workspaces/ folder where teams can create a folder of the style workspace<team> to store their code. This code will not be reviewed by the Data Team for style. The only concern given prior to merge is whether it runs and if there are egregious inefficiencies in the code that could impact production runs.

To add a new space:

- Create an issue in the analytics project and open a new merge request
- Create a new folder in /models/workspaces/ e.g. workspacesecurity

- Add an entry to the dbtproject.yml file for the new workspace. Include the schema it should write to:

```
yaml
 ------------------
 Workspaces
 ------------------
workspaces:
  +tags: ["workspace"]

  workspacesecurity:  This maps to the folder in /models/workspaces/
    +schema: workspacesecurity  This is the schema in the prod database
```

- Add your .sql files to the folder
- Add any entries to the CODEOWNERS file
- Use the dbt Workspace Changes MR template and follow the instructions there to submit the MR for review and final merge

Newly added code will take up to 24 hours to appear in the data warehouse.

The Data Team reservers the right to reject code that will dramatically slow the production dbt run. If this happens, we will consider building a separate dbt execution job just for the workspaces.

General

- Model names should be as obvious as possible and should use full words where possible, e.g. accounts instead of accts. Avoid using aliases unless absolutely necessary. Table names in the warehouse should match the dbt model names to maintain clarity, consistency, and ease of debugging. If an alias is required, document the rationale clearly in the model's description.

- Documenting and testing new data models is a part of the process of creating them. A new dbt model is not complete without tests and documentation.

- Follow the naming convention of analysis type, data source (in alpha order, if multiple), thing, aggregation

```
sql
-- Good
retentionsfdczuoracustomercount.sql

-- Bad
retention.sql
```

- All {{ ref('...') }} statements should be placed in CTEs at the top of the file. (Think of these as import statements.)
  - This does not imply all CTE's that have a {{ ref('...') }} should be SELECT only. It is ok to do additional manipulations in a CTE with a ref if it makes sense for the model.

  - If only a small number of fields are required from a model containing many columns then it can be performant to list them in the CTE, otherwise it is better to use SELECT . To do this, the simplecte macro can be used.

- If you want to separate out some complex SQL into a separate model, you absolutely should to keep things DRY and easier to understand. The config setting materialized='ephemeral' is one option which essentially treats the model like a CTE.

Model Configuration

There are multiple ways to provide configuration definitions for models.
The dbt docs for configuring models provide a concise explanation of the ways to configure models.

Our guidelines for configuring models:

- The default materialization is view
- The default schema is prep.preparation.
- Disabling any model should always be done in the dbtproject.yml via the +enabled: false declaration
- Configs should be applied in the smallest number of locations:
  - If <50% of models in a directory require the same configuration, then configure the individual models
  - If >=50% of models in a directory require the same configuration, strongly consider setting a default in the dbtproject.yml, but think about whether that setting is a sensible default for any new models in the directory

Versions

Model versions within dbt allow for a controlled transition between model changes that may break or significantly impact downstream uses. When versions are defined for a model all enabled versions will be produced in the target database and schema. This allows users of the model to access the versions side by side so that any breaking changes can be addressed while still delivering reports and analysis using an earlier version. This is most effective when columns are removed or significant refactor of several models is taking place over multiple development cycles.

Defining Model Versions

Model versions are implemented by creating a new model file with the same name as the target model but with an added version suffix, and by defining version properties in the relevant schema.yml file. More details can be found in the dbt documentation for model versions.

yml
 dimdate.sql
 dimdatev2.sql

models:
  name: dimdate
  ...

```
versions:
  - v: 1
    definedin: dimdate.sql  Only needed if there is no suffix on the model file.
  - v: 2
latestversion: 1
```

Referencing Model Versions

With the provided model definitions, two models will be created when the model is included in a selection, such as dbt run --models dimdate; database.schema.dimdatev1 and database.schema.dimdatev2. Additionally, with the createlatestversionview post-hook, a view will be created of the latest version; database.schema.dimdate. This view allows users to always be on the latest version of a model without needing to know what that version is.

Within dbt, if a specific version of a model needs to be used to build an other model, it can be defined in the ref function:

jinja
```
{{ ref('dimdate', v=2) }}
```

If a version is not defined in the ref function then the latest version of the model will be used.

Depends On

In normal usage, dbt knows the proper order to run all models based on the usage of the {{ ref('...') }} syntax. There are cases though where dbt doesn't know when a model should be run. A specific example is when we use the schemaunionall or schemaunionlimit macros. In this case, dbt thinks the model can run first because no explicit references are made at compilation time. To address this, a comment can be added in the file, after the configuration setting, to indicate which model it depends on:

sql
```
{{config({
    "materialized":"view"
  })
}}

-- dependson: {{ ref('snowplowsessions') }}

{{ schemaunionall('snowplow', 'snowplowsessions') }}
```

dbt will see the ref and build this model after the specified model.

Database and Schema Name Generation

In dbt, it is possible to generate custom database and schema names. This is used extensively in our project to control where a model is materialized and it changes depending on production or development use cases.

Databases

The default behavior is documented in the "Using databases" section of the dbt documentation. A macro called generatedatabasename determines the schema to write to.

We override the behavior of this macro with our own generatedatabasename definition. This macro takes the configuration (target name and schema) supplied in the profiles.yml as well as the schema configuration provided in the model config to determine what the final schema should be.

Schemas

The default behavior is documented in the "Using custom schemas" section of the dbt documentation. A macro called generateschemaname determines the schema to write to.

We override the behavior of this macro with our own generateschemaname definition. This macro takes the configuration (target name and schema) supplied in the profiles.yml as well as the schema configuration provided in the model config to determine what the final schema should be.

Development behavior

In FY21-Q4, we switched to having development databases instead of schemas. This means that the schemas that are used in production match what is used in development, but the database location is different. dbt users should have their own scratch databases defined, such as TMURPHYPROD and TMURPHYPREP, where models are written to.

This switch is controlled by the target name defined in the profiles.yml file. Local development should never have prod or ci as the target.

Macros

Naming conventions

- File name must match the macro name

Structure

- Macros should be documented in either the macros.yml file or in a macros.md file in descriptions are long
- Use the arguments property in macros.yml to describe the input variables

dbt-utils

In our dbt project we make use of the dbt-utils package. This adds several macros that are commonly useful. Important ones to take note of:

- groupby - This macro build a group by statement for fields 1...N

- star - This macro pulls all the columns from a table excluding the columns listed in the except argument
- surrogatekey - This macro takes a list of field names and returns a hash of the values to generate a unique key

Seeds {seeds}

Seeds are a way to load data from csv files into our data warehouse (dbt documentation). Because these csv files are located in our dbt repository, they are version controlled and code reviewable.
This method is appropriate for loading static data which changes infrequently.
A csv file that's up to ~1k lines long and less than a few kilobytes is probably a good candidate for use with the dbt seed command.
A seed file should be placed in the project folder that corresponds to the functional team that has ownership of the information found therein. This folder structure also corresponds to a schema in the PREP database so that the data can be easily used in further development.

Organizing columns

When writing a base model, columns should have some logical ordering to them.
We encourage these 4 basic groupings:

- Primary data
- Foreign keys
- Logical data - This group can be subdivided further if needed
- Metadata

Primary data is the key information describing the table. The primary key should be in this group along with other relevant unique attributes such as name.

Foreign keys should be all the columns which point to another table.

Logical data is for additional data dimensions that describe the object in reference. For a Salesforce opportunity this would be the opportunity owner or contract value. Further logical groupings are encouraged if they make sense. For example, having a group of all the variations of contract value would make sense.

Within any group, the columns should be alphabetized on the alias name.

An exception to the grouping recommendation is when we control the extraction via a defined manifest file. A perfect example of this is our gitlab.com manifest which defines which columns we extract from our application database. The base models for these tables can be ordered identically to the manifest as it's easier to compare diffs and ensure accuracy between files.

- Ordered alphabetically by alias within groups

```
sql
-- Good

SELECT
```

```
  id                  AS accountid,
  name                AS accountname,

  -- Foreign Keys
  ownerid             AS ownerid,
  pid                 AS parentaccountid,
  zid                 AS zuoraid,

  -- Logical Info
  opportunityownerc  AS opportunityowner,
  accountownerc      AS opportunityownermanager,
  ownerteamoc        AS opportunityownerteam,

  -- metadata
  isdeleted           AS isdeleted,
  lastactivitydate    AS lastactivitydate
FROM table
```

- Ordered alphabetically by alias without groups

```
sql
-- Less Good

SELECT
  id                  AS accountid,
  name                AS accountname,
  isdeleted           AS isdeleted,
  lastactivitydate    AS lastactivitydate,
  opportunityownerc  AS opportunityowner,
  accountownerc      AS opportunityownermanager,
  ownerteamoc        AS opportunityownerteam,
  ownerid             AS ownerid,
  pid                 AS parentaccountid,
  zid                 AS zuoraid
FROM table
```

- Ordered alphabetically by original name

```
sql
-- Bad

SELECT
  accountownerc      AS opportunityownermanager,
  id                  AS accountid,
  isdeleted           AS isdeleted,
  lastactivitydate    AS lastactivitydate
  name                AS accountname,
  opportunityownerc  AS opportunityowner,
```

```
    ownerteamoc       AS opportunityownerteam,
    ownerid           AS ownerid,
    pid               AS parentaccountid,
    zid               AS zuoraid
  FROM table
```

Tags

Tags in dbt are a way to label different parts of a project. These tags can then be utilized when selecting sets of models, snapshots, or seeds to run.

Tags can be added in YAML files or in the config settings of any model. Review the dbtproject.yml file for several examples of how tags are used. Specific examples of adding tags for the Trusted Data Framework are shown below.

Within the analytics and data-tests projects we enforce a Single Source of Truth for all tags. We use the Valid Tags CSV to document which tags are used. Within merge requets, there is a validation step within every dbt CI job that will check this csv against all tags used in the project and fail the job if there is a mismatch. In the future, we aim to include more metadata about tags within this csv file.

Be aware that tags applied at any level do not apply to any tests. Tags for tests have to be explicitly applied for every test within the schema.yml file.

Warehouse Size

Configuring the warehouse size within the model can be a way to ensure needed performance on a model. This is intended to be performed on a model by model bases and for models with known performance needs; for example, the model contains a large recursive CTE that will fail if not run on a LARGE or bigger warehouse.

This configuration can be done using the generatewarehousename macro within the model configuration and is designed to work with both production, TRANSFORMER, and development DEV warehouses. To configure the warehouse size, you need to pass the desired size to the macro where the correct warehouse name will be generated. This will only work for currently existing warehouses.

```
jinja
{{
  config(
    snowflakewarehouse = generatewarehousename('XL')
  )
}}
```

Sample Data in Development

To streamline local development on local models, a way to sample, or use a subset of the data, is made available to the developers. This tool will allow developers the option of using sample

data, or full data depending on what the situation calls for, allowing them to iterate quickly on the structure of models using sample data and then switch to full data when validation is needed. Using this in conjunction with local cloning of tables should improve the developer cycle time.

When selecting the tool to use, the developer should consider speed gained from the tool and the consequence of leaving the sampling in the code. A Random Sample is easy to add to the code, but if left in the code base, it puts the quality of the production data at risk, whereas a Sample Table will take longer to set up, but has no risk to the production data.

Random Sample

With the macro sampletable the developer will be able to select a random percent of the target table. This macro takes a percentage value which represent the amount of the target table to be returned. While the sample is random it is also deterministic, meaning that if the target table has not changed each query will return the same rows. If this macro is left in the code it will be executed in production and ci environments so it should be removed before merging in to the main code branch.

Example Use:

sql

```
SELECT
FROM {{ ref('dimdate') }} {{ sampletable(3) }}

-- Resolve to:
SELECT
FROM devprod.common.dimdate SAMPLE SYSTEM (3) SEED (16)
```

Sample Table

With a sample table the developer creates a table that represents a sub set of the data in the target table that existing models will use instead of the original table. This requires configuring the desired samples and performing operations to create the sample tables. When configuring the sample the sample clause can use the sampletable macro or any filtering statement that can be used after the FROM statement such as LIMIT, WHERE, or QUALIFY. These sample tables will only be used in non production and ci environments and will not appear in the model lineage.

Workflow steps:

- Clone the target sample tables
  - Using a command such as clone-dbt-select-local-user ensure that there is a full data table to sample from.
- Configure the sample for each table to be created
  - Samples are configured in the samples macro in the samplesyml variable

yml

```
samples:
  - name: dimdate
    clause: "{{ sampletable(3) }}"
```

Or

```
samples:
  - name: dimdate
    clause: "WHERE dateactual >= DATEADD('day', -30, CURRENTDATE())"
```

- Construct the sample tables
  - Using the run-operation command execute the createsampletables macro

```
console
dbt run-operation createsampletables
```

- Develop and iterate as needed
- Final test run using the full data
  - Override the localdata variable to be full-data as part of the dbt execution

```
console
dbt run -s dimdate --vars 'localdata: fulldata'
```

- Remove sample configuration
  - Remove the list of samples from the samplesyml before merging the code changes

Example Use:

sql

```
-- Sample configuration
/
samples:
  - name: dimdate
    clause: "WHERE dateactual >= DATEADD('day', -30, CURRENTDATE())"
/

-- dbt run-operation createsampletables executes the following command
CREATE OR REPLACE TRANSIENT TABLE devprod.common.dimdatesample AS SELECT  FROM
devprod.common.dimdate WHERE dateactual >= DATEADD('day', -30, CURRENTDATE());

-- In model ref function will retrieve the sample table
SELECT
FROM {{ ref('dimdate') }}
```

```
-- Resolves to:
SELECT
FROM devprod.common.dimdatesample
```

How Sampling Macros Work

For more details on how the macros used in sampling function see the following documentaion:

- createsampletables
- generatesampletablesql
- getsamplerelation
- istablesampled
- sampletable
- samples
- ref

Trusted Data Framework

See the Trusted Data Framework section of our Platform page for a deeper dive into the philosophy behind the Trusted Data Framework.

Schema To Golden Data Coverage

We implement 12 categories of Trusted Data Framework (TDF) monitors and tests (monitors are created in and executed by Monte-Carlo, tests are created with and executed by dbt):

1. Freshness monitors Monitor for unusual delays in table and field updates
1. Schema monitors Monitor fields that are added, removed or changed
1. Volume monitors Monitor for unusual changes in table size based on the numbers of rows
1. Field health Monitor Monitor fields for dips or spikes in stats like % null, % unique, and more. Our ML sets the thresholds.
1. SQL rule monitor Write a SQL statement to check for any expressable condition across 1 or more tables in your data.
1. JSON schema monitor Monitor for schema changes in JSON data added to a table field.
1. Dimension tracking Monitor for changes in the distribution of values within a low-cardinality table field.
1. Schema tests to validate the integrity of a schema
1. Column Value tests to determine if the data value in a column matches pre-defined thresholds or literals
1. Rowcount tests to determine if the number of rows in a table over a pre-defined period of time match pre-defined thresholds or literals
1. Custom SQL tests any valid SQL that doesn't conform to the above categories

Our tests are stored in 2 primary places - either in a YAML file within our main project or within our Data Tests project.

Schema and Column Value tests will usually be in the main project. These will be in schema.yml

and sources.yml files in the same directory as the models they represent.

Rowcount, and any other custom SQL tests will always be in the Data Tests project. This is a private project for internal GitLab use only.

Tagging

Tagging the tests is an important step in adding new tests. Labeling the test with a dbt tag is how we parse and identify tests when building trusted data dashboards. There are 2 ways to tag tests depending on their type.

The first is by adding tags in the YAML definition. This can be done at the highest level of the YAML definition for source tests, or on the column level for model tests.

yaml
Source Labeling in sources.yml

```
version: 2

sources:
  - name: zuora
    tags: ["tdf","zuora"]
```

Model Labeing in schema.yml

```
version: 2

models:
  - name: zuoraaccountingperiodsource
    description: Source layer for Zuora Accounting Periods for cleaning and renaming
    columns:
      - name: accountingperiodid
        tags: ["tdf","zuora"]
        tests:
          - notnull
          - unique
```

Each of these examples will apply the tags to all tests nested in the underlying hierarchy.

The second way of adding tags is via the config declaration at the top of a test file:

sql

```
{{ config({
    "tags": ["tdf","zuora"]
    })
}}

WITH test AS (...)
```

General Guidance

- Every model should be tested in a schema.yml file
- At minimum, unique fields, not nullable fields, and foreign key constraints should be tested (if applicable)
- The output of dbt test should be pasted into MRs
- Any failing tests should be fixed or explained prior to requesting a review

Schema Tests

Schema tests are designed to validate the existence of known tables, columns, and other schema structures. Schema tests help identify planned and accidental schema changes.

All Schema Tests result in a PASS or FAIL status.

Schema Test Example

Purpose: This test validates critical tables exist in the Zuora Data Pipeline.

We've implemented schema tests as a dbt macro. This means that instead of writing SQL, a user can add the test by simply calling the macro. This is controlled by the rawtableexistence macro.

sql
```
-- File: https://gitlab.com/gitlab-data/analytics/-/blob/master/transform/snowflake-dbt/tests/sources/zuora/existence/zuorarawsourcetableexistence.sql
{{ config({
    "tags": ["tdf","zuora"]
    })
}}

{{ rawtableexistence(
    'zuorastitch',
    ['account', 'subscription', 'rateplancharge']
) }}
```

Column Value Tests

Column Value Tests determine if the data value in a column is within a pre-defined threshold or matches a known literal. Column Value Tests are the most common type of TDF test because they have a wide range of applications. Column Value tests are useful in the following scenarios:

- change management: pre-release and post-release testing
- ensuring sums/totals for important historical data meets previously reported results
- ensuring known "approved" data always exists

Column value tests can be added as both YAML and SQL. dbt natively has tests to assert that a column is not null, has unique values, only contains certain values, or that all values in a column are represented in another model (referential integrity).

We also use the dbt-utils package to add even more testing capabilities.

All Column Value Tests result in a PASS or FAIL status.

Column Value Test Example 1

Purpose: This test validates the account ID field in Zuora. This field is always 32 characters long and only has numbers and lowercase letters.

Because we use dbt, we have documentation for all of our source tables and most of our downstream modeled data. With in the yaml documentation files, we're able to add tests to individual columns.

yaml
File: https://gitlab.com/gitlab-data/analytics/-/blob/master/transform/snowflake-dbt/models/sources/zuora/sources.yml

```
    - name: account
    description: '{{ doc("zuoraaccountsource") }}'
    columns:
        - name: id
        description: Primary Key for Accounts
        tags: ["tdf","zuora"]
        tests:
            - dbtutils.expressionistrue:
                expression: "id REGEXP '[0-9a-z]{32}'"
```

Rowcount Tests

The Rowcount test is a specialized type of Column Value test and is broken out because of its importance and utility. Rowcount tests determine if the number of rows in a table over a period of time meet expected pre-defined results. If data volumes change rapidly for legitimate reasons, rowcount tests will need to be updated appropriately.

Rowcount Test Example 1

Purpose: This test validates we always had 18,849 Zuora subscription records created in 2019.

This test is implemented as a dbt macro. This means that instead of writing SQL, a user can add the test by simply calling the macro. This is controlled by the sourcerowcount macro.

sql
-- https://gitlab.com/gitlab-data/data-tests/-/blob/main/tests/sources/zuora/rowcount/zuorasubscriptionsourcerowcount2019.sql

```
{{ config({
    "tags": ["tdf","zuora"]
    })
}}

{{ sourcerowcount(
    'zuora',
    'subscription',
```

```
    18489,
    "autorenew = 'TRUE' and createddate > '2019-01-01' and createddate < '2020-01-01'"
) }}
```

Rowcount Test Example 2

Purpose: We have a fast-growing business and should always have at least 50 and at most 200 new Subscriptions loaded from the previous day. This is controlled by the modelnewrecordsperday macro.

```
sql
-- https://gitlab.com/gitlab-data/data-tests/-/blob/main/tests/sources/zuora/rowcount/zuorasubscriptionsourcemodelnewrecordsperday.sql
{{ config({
    "tags": ["tdf","zuora"],
    "severity": "warn",
    })
}}

{{ modelnewrowsperday(
    'zuorasubscriptionsource',
    'createddate',
    50,
    200,
    "datetrunc('day',createddate) >= '2020-01-01'"
) }}
```

Custom SQL

You may have a test in mind that doesn't fit into any of the above categories. You can also write arbitrary SQL as a test. The key point to keep in mind is that the test is passing if no rows are returned. If any rows are returned from the query, then the test would fail.

An example of this from the dbt docs:

```
sql
{{ config({
    "tags": ["tdf","fctpayments"]
    })
}}

-- Refunds have a negative amount, so the total amount should always be >= 0.
-- Therefore return records where this isn't true to make the test fail
SELECT
    orderid,
    sum(amount) AS totalamount
```

```
FROM {{ ref('fctpayments' )}}
GROUP BY 1
HAVING totalamount < 0
```

Any valid SQL can be written here and any dbt models or source tables can be referenced.

Merge Request Workflow

There are a few scenarios to think about when adding or updating tests.

The first scenario is modifying or adding a test in a YAML file within our main project. This follows our standard MR workflow and nothing is different. Run the CI jobs as you normally would.

The second scenario is adding any test or golden data records within the data-tests project against tables that are being updated or added via an MR in the analytics project. This is the most common scenario. In this case, no pipelines need to be executed in the data-tests project MR. The regular dbt pipelines in the analytics MR can be run and the only change is the branch name of the data-tests project needs to be passed to the job via the DATATESTBRANCH environment variable.

The third scenario is when tests are being added to the data-tests project, but no golden data CSV files are being updated or added, AND there is no corresponding MR in the analytics project. In this scenario, you will see some CI jobs that will run your tests against production data. This is useful for ensuring minor changes (such as syntax, tags, etc.) work.

The fourth scenario is when golden data CSV files are being added or updated and there is no corresponding analytics MR. In this case, we do not want to test against production as the golden data files are inserted into the database as tables. In this scenario, you will see additional CI jobs. There are some to create a clone of the warehouse, some to run dbt models stored in the analytics project against the clone, and some to run the tests against the clone.

This flowchart should give a rough guide of what to do. Follow the instructions in the relevant MR template in the project for more detailed instructions.

```
mermaid
graph TD
  A[Do you want to add YAML or SQL tests?] -->|YAML| B[Open an Analytics MR <br/>and proceed as usual]
  A -->|SQL| C[Open a Data Tests MR]
  A -->|Both| D[Open an Analytics and Data Tests MR]


  D --> E[Run dbt jobs as usual in analytics <br/>but provide the branch name of the <br/> Data Tests project as DATATESTBRANCH]

  C --> F[Are you updating or adding<br/> a Golden Data CSV?]

  F -->|Yes| G[Run a clone - full or shallow - then run dbt jobs]
```

F -->|No| H[Run dbt jobs against production]

In the case where you have a merge request in data-tests and one in analytics, the analytics MR should be set as a dependency of the data-tests MR. This means that the analytics MR must be merged prior the data-tests MR being merged.

Running the newly introduced dbt tests in the data-tests project

Steps to follow in order to run the tests you implemented in the data-tests project from your machine, while developing them:

1. Push your changes to the remote branch you are working on the data-tests project
2. Go to your analytics project locally, create a new branch (git checkout -b <branchname>) with the same name as the one at data-tests & modify the Makefile to edit the DATATESTBRANCH to match your branch name on the data-test project
3. From the analytics project run make run-dbt
4. You should see some logs, which also show the revision data-tests was installed from, where you should see your branch
5. From where you currently are (which should be the snowflake-dbt directory) run the corresponding command for testing your own model

Example

To run the zuorarevenuerevenuecontractlinesource rowcount tests, we can use the following command, which should work without any issues:

dbt --partial-parse test --models zuorarevenuerevenuecontractlinesource

> :warning: Please note, whenever you make changes to the underlying tests in the data-tests project, you need to push those changes to the remote and re-run steps 3-5, to start a dbt container with the latest changes from your branch.

Data extraction (RAW data layer)

Data extraction is loading data from the source system to Snowflake data warehouse in the RAW data layer.

Data transformation (Prod data layer)

Data transformation is downstream transformation via dbt for Dimensions, Facts, Marts and reports models.

Snapshots {snapshots}

dbt snapshots are SCD Type 2 tables, built on top of mutable (SCD Type 1) source tables that record changes to the source table over time.

This single snapshot table, due to its SCD Type 2 nature, captures the entire history of changes in the source table.

For more on snapshots, including examples, go to dbt docs.

Take note of how we talk about and define snapshots.

Create snapshot tables with dbt snapshot

Snapshot definitions are stored in the snapshots folder of our dbt project.

We have organized the different snapshots by data source for easy discovery.

The following is an example of how we implement a snapshot:

sql

```
{% snapshot sfdcopportunitysnapshots %}

   {{
      config(
        uniquekey='id',
        strategy='timestamp',
        updatedat='systemmodstamp',
      )
   }}

   SELECT
   FROM {{ source('salesforce', 'opportunity') }}

{% endsnapshot %}
```

Snapshot best practices

- Database and Schema Configuration: Configure the database and schema in dbtproject.yml. Use an environmental variable for the database and set the schema to snapshots. This ensures consistency and simplifies deployment across environments.
- Follow Naming Conventions: The table name in the data warehouse should follow the {sourcetablename}snapshots naming convention.
- Avoid Transformations: Perform minimal transformations in snapshot models aside from deduplication. Cleaning and transformation logic should be handled downstream to maintain snapshot simplicity.
- Prefer Timestamp Strategy: Unless a reliable updatedat field is unavailable, prefer the timestamp strategy over check. However, note that in Salesforce, the SystemModstamp field does not capture changes to formula fields. For SFDC snapshots, it·s better to use the check strategy and validate all columns to ensure no updates are missed. Refer to the dbt documentation for more details on snapshot strategies.
- Enable invalidateharddeletes: Use the invalidateharddeletes option for snapshots where it·s critical to track and exclude deleted records. With this setting enabled, records deleted from the source are assigned a valid end timestamp (dbtvalidto) instead of leaving it NULL.

Snapshot Model Types

A dbt Snapshot model is designed to capture changes to records for a single table over time, providing a historical view of the data. The table being snapshotted can originate from a source table or an existing table already used for analysis. Snapshots are defined using the {% snapshot tablename %} configuration in a snapshot file, which specifies how changes are tracked and stored.

dbt Snapshot Model Strategy

The strategy to determine when a new snaphot record is written can be configured 2 different ways:

- timestamp uses an updatedat column to determine if a row has changed since the last snapshot was taken
- check uses a list of columns to determine if any of the columns have changed since the last time the snapshot was taken.

The record version is determined by the dbtvalidfrom and dbtvalidto columns. These TIMESTAMP columns are created automatically by dbt and utilized by the snapshot model to determine the timeframe of each snapshotted row.

- When a new snapshot record is written, dbtvalidfrom is assigned the current date and time, and dbtvali